Closing the Gap to MonoCera with Trace Lemmas

Case Studies on Array Programs from SV-COMP

Experiments with Cuvée 0.35.15

Abstract

MonoCera (Amilon et al., CAV 2023) verifies array programs by instrumenting aggregation and quantification operators. On the 125 SV-COMP tests of its evaluation, Cuvée with trace equation systems and simplification proved 51 tests in a baseline run, against 42 for MonoCera, but missed almost all tests over sums. We report on lemmas, kept in a separate file and injected by `-include`, that close 22 further tests, among them all 17 tests over sums and six tests that only MonoCera had proved. We present case studies, the lemmas that are key to them, and the reasoning steps by which the simplifier closes the goals.

1 Setting

Cuvée translates each loop of a C program into a *trace equation*: the values of a variable at the loop head form a trace

$$X = \mathit{trace}(x_0, \textbf{for } D \textbf{ do } x, d :: u),$$

whose dependencies D are traces of the same length, e.g. the counter $\lfloor \mathit{counter}(0,1,n) \rfloor = 0, 1, \ldots, n-1$ of the iterations, and whose update u may consist of guarded cases. The value after the loop is $\mathit{current}(X)$. Assertions in a later loop become goals over $\mathit{current}(X)$, where the assertions of the earlier iterations of the checking loop are hypotheses. The simplifier rewrites the goals with a database of lemmas (`teq.dfy`); lemmas are rewrite rules $\ell = r$, possibly with side conditions, whose left-hand sides may contain higher-order variables, e.g. $G(k)$, which match by abstraction (Miller's fragment).

The tests are the SV-COMP programs of the MonoCera artifact, converted as for the other tools of its evaluation (macros substituted, `malloc` replaced by arrays), keeping the SV-COMP harness. The lemmas were developed in a separate file, `lemmas.dfy`; the generic ones have since moved into `teq.dfy`, with proofs in Lean, the specific ones (strides, stores ahead, `half_even`) are injected from it; a run is

```
./Cuvee.sh -teq -simplify -split -simplify \
  -include benchmarks/monocera/lemmas.dfy <test>.c
```

For each closed test we checked a variant with a false assertion, which stays open: the lemmas compute the value in question, they do not make the goal vacuous.

2 A Sum in an Array Cell

The test `sum/brs1` adds the elements of an array, except the first, into the cell 0 of a one-element array:

```
int sum[1]; int a[N];
for (i = 0; i < N; i++) a[i] = (i % 1 == 0) ? 1 : 0;
for (i = 0; i < N; i++)
  if (i == 0) sum[0] = 0; else sum[0] = sum[0] + a[i];
__VERIFIER_assert(sum[0] <= N);
```

Open goal. The first loop is already resolved ($a[i] = 1$). The accumulator is not a scalar but an array cell, hence its loop is a trace of *stores*:

```
Sum == trace(sum0, for ⌊K⌋, ⌊I⌋ do sum, k, i ::
         case k == 0 => sum[0 := 0]
         case 0 < k  => sum[0 := sum[0] + 1])
  ==>  current(Sum)[0] <= N
```

The lemma `current_trace_reset_add` of `teq.dfy` solves this pattern for a scalar, which is how a hand translation to Dafny with a scalar `sum` is proved, but it does not apply to a store.

Key lemma. Only the cell j is written, hence reading it from the final array is the scalar sum:

```
lemma {:rw} select_current_trace_reset_add(
    a0: array<int, int>, j: int, n: Nat, c: int,
    G: (int) -> int)
  ensures current(trace(a0,
      for ⌊counter(0,1,n)⌋, ⌊counter(0,1,n)⌋ do a, k, i ::
        case k == 0 => a[j := c]
        case 0 < k  => a[j := a[j] + G(k)]))[j]
    == if n == zero then a0[j]
       else c + sum(maps(for ⌊counter(0,1,n)⌋ yield k ::
                           if 0 < k then G(k) else 0))
```

The right-hand side is the one of `current_trace_reset_add`, such that `teq.dfy` continues from there.

Reasoning steps.

(1) The lemma matches with $j = 0$, $c = 0$, and $G = (z) \mapsto 1$, which the higher-order variable abstracts from the summand.
(2) Since $N > 0$, $n \neq \mathit{zero}$, and the cell is $0 + \mathit{sum}(\ldots)$.
(3) `teq.dfy` sums the constant summands: the sum is $n - 1$.
(4) With $\mathit{integer}(n) = N$, the goal $N - 1 \leq N$ holds.

A variant asserting `sum[0] <= N - 2` stays open, with the remaining goal $\neg(0 < N)$: the lemma computes the value $N - 1$ rather than making the goal vacuous.

A bug found on the way. In `brs2`, the summand is $a[k]$, and G was instantiated with the array a itself: the simplifier's eta reduction turned the instance $(z) \mapsto a[z]$, of function type, into the array a, so that $G(k)$ remained an application $a(k)$ of the array, opaque to all rules about $a[k]$. Restricting eta reduction to lambdas of array type fixed this (version 0.35.15).

3 An Update in Place

The test `sum/conda` sums an array after a second loop has modified it in place, by cases on the old value:

```
for (i = 0; i < N; i++) a[i] = 1;
for (i = 0; i < N; i++)
  if (a[i] == 1) a[i] = a[i] + 1;
  else a[i] = a[i] - 1;
for (i = 0; i < N; i++) sum[0] = sum[0] + a[i];
__VERIFIER_assert(sum[0] == 2 * N);
```

Open goal. `teq.dfy` already turns the sum into $sum(selects(a, \lfloor I \rfloor))$, but the array is the current of a trace whose update reads the cell it writes:

```
A == trace(a1, for ⌊K⌋, ⌊I⌋ do a, k, i ::
       case a[k] == 1    => a[k := 2]
       case !(a[k] == 1) => a[k := a[k] - 1])
```

Key lemma. Each position k is written once, and read only in its own iteration, before it is written; hence the loop stores values computed from the *initial* array:

```
lemma {:rw} stores_cases_def(
    a0: array<int, int>, n: Nat, c: int,
    P: (int) -> bool, G: (int) -> int)
  ensures trace(a0,
      for ⌊counter(0,1,n)⌋, ⌊counter(0,1,n)⌋ do a, k, i ::
        case P(a[k])  => a[k := c]
        case !P(a[k]) => a[k := G(a[k])])
    == stores(a0, ⌊counter(0,1,n)⌋,
         maps(for ⌊counter(0,1,n)⌋ yield k ::
                if P(a0[k]) then c else G(a0[k])))
```

The first case stores a constant: a higher-order instance must contain the term it abstracts, here $a[k]$, so $G(a[k])$ cannot match the constant 2.

Reasoning steps.

(1) The lemma matches with $P = (z) \mapsto z = 1$, $c = 2$, and $G = (z) \mapsto z - 1$.
(2) `selects_stores_counter` of `teq.dfy` reads the stored values back at the positions of the counter.
(3) $a_0[k] = 1$ by the first loop, so each stored value is 2.
(4) The sum of n twos is $2n = 2N$.

The same lemma closes `s1lif-s5lif`, whose second loops have the same shape.

4 Values Chosen by the Index

The test `sum/brs2` sums an array whose elements alternate between 2 and 0, and asserts an upper bound:

```
for (i = 0; i < N; i++) a[i] = (i % 2 == 0) ? 2 : 0;
for (i = 0; i < N; i++)
  if (i == 0) sum[0] = 0; else sum[0] = sum[0] + a[i];
__VERIFIER_assert(sum[0] <= 2 * N);
```

Key lemmas. Three lemmas cooperate. The cell lemma of Section 2 turns the accumulator into a sum over $a[k]$. To read $a[k]$, the first loop, which stores constants chosen by cases on the *index*, becomes a trace of stores:

```
lemma {:rw} stores_index_cases_def(
    a0: array<int, int>, n: Nat, c: int, d: int,
    P: (int) -> bool)
  ensures trace(a0,
      for ⌊counter(0,1,n)⌋, ⌊counter(0,1,n)⌋ do a, k, i ::
        case P(k) => a[k := c] case !P(k) => a[k := d])
    == stores(a0, ⌊counter(0,1,n)⌋,
         maps(for ⌊counter(0,1,n)⌋ yield k ::
                if P(k) then c else d))
```

Finally, a sum of n terms that are each 0, c, or d is bounded:

```
lemma {:rw} sum_cases_below(n: Nat, c: int, d: int,
    e: int, P: (int) -> bool)
  requires 0 <= e && integer(n) * c <= e
       && integer(n) * d <= e
  ensures sum(maps(for ⌊counter(0,1,n)⌋ yield k ::
      if 0 < k then (if P(k) then c else d) else 0))
    <= e
```

Its left-hand side is a comparison, whose constants bind the bound e. The general `sum_below_prefix` of `teq.dfy` does not apply, since its bound occurs only in a side condition, which would have to be among the facts.

Reasoning steps.

(1) The cell lemma yields $sum(maps(\ldots \textbf{ if } 0 < k \textbf{ then } a[k] \textbf{ else } 0))$.
(2) The index lemma, with $P = (z) \mapsto z \bmod 2 = 0$, $c = 2$, and $d = 0$, makes a the current of stores, and `current_stores` of `teq.dfy` reads $a[k] = \textbf{if } k \bmod 2 = 0 \textbf{ then } 2 \textbf{ else } 0$.
(3) The bound lemma with $e = 2N$ leaves the side conditions $0 \le 2N$, $2n \le 2N$, and $0 \le 2N$, which hold by $integer(n) = N$.

The same lemmas close `brs3-brs5`. A bound $sum(\ldots k \bmod m \ldots) \le n(m-1)$ for $0 < m$ closes `ms2-ms5`, whose arrays hold $i \bmod m$.

5 Strided Writes

The test `forall/pr2` fills an array of even size pairwise, with values that depend on a bound `MINVAL`:

```
if (CELLCOUNT % 2 != 0) return 1;
for (i = 1; i <= CELLCOUNT / 2; i++) {
  volArray[2*i - 2] = (3 >= MINVAL) ? 3 : 0;
  volArray[2*i - 1] = (1 >= MINVAL) ? 1 : 0;
}
for (i = 0; i < CELLCOUNT; i++)
  __VERIFIER_assert(volArray[i] >= MINVAL
                    || volArray[i] == 0);
```

Key lemma. Each iteration writes the pair $2k$, $2k+1$; the pairs are disjoint and cover $0, \ldots, 2n-1$. The update consists of three cases on `MINVAL`, which does not change in the loop:

```
lemma {:rw} select_current_trace_stride2_cases(..)
  requires R == (!P && !Q)
  ensures current(trace(a0,
      for ⌊counter(0,1,n)⌋, ⌊counter(1,1,n)⌋ do a, k, i ::
        case P => a[2*k := X1][2*k + 1 := Y1]
        case Q => a[2*k := X2][2*k + 1 := Y2]
        case R => a[2*k := X3][2*k + 1 := Y3]))[j]
    == if 0 <= j && j < 2 * integer(n)
       then (if j % 2 == 0
             then (if P then X1 else if Q then X2 else X3)
             else (if P then Y1 else if Q then Y2 else Y3))
       else a0[j]
```

The side condition requires the guards to be exhaustive; otherwise the update would be unspecified where none holds, and the equation unsound.

Reasoning steps. After `-split`, three cases arise.

(1) A case with the contradictory hypotheses $1 < CELLCOUNT$ and $CELLCOUNT/2 \le 0$, refuted by $x/c \le 0 \iff x < c$ for $0 < c$ (`div_nonpos`).
(2) Within $0 \le i < 2n$, the stride lemma yields the value by parity and guards, each of which satisfies the assertion.
(3) The case $2n \le i < CELLCOUNT$ is empty, since $n = CELLCOUNT/2$ and $CELLCOUNT$ is even.

A limitation of the simplifier. The third case stays open: the simplifier inlines solutions of arithmetic equations into polynomials only, and does not relate n to the atom $CELLCOUNT/2$. A lemma works around it by stating the half of an even number without a quotient:

```
lemma {:rw} half_even(n: int, x: int)
  requires x % 2 == 0
```

```
  ensures (n == x / 2) == (x == 2 * n)
```

Then $CELLCOUNT = 2n$ is a polynomial equation, which the simplifier solves and substitutes, and $2n \le i < 2n$ is empty. The constant strides 2 and 3 close `array_doub_access_init_const` and `array_tripl_access_init_const` without further steps.

6 Stores Ahead

The test `forall/rewnif` writes one cell ahead before it writes the current one:

```
for (i = 0; i < SIZE; i++) {
  if (i + 1 < SIZE) a[i+1] = i;
  a[i] = i;
}
for (i = 0; i < SIZE; i++)
  __VERIFIER_assert(a[i] >= i);
```

Key lemma. The value written ahead at $k + 1$ is overwritten by the next iteration, so the cell $j < n$ holds what iteration j writes last:

```
lemma {:rw} select_current_trace_ahead(
    a0: array<int, int>, n: Nat, m: int,
    F: (int) -> int, G: (int) -> int, j: int)
  requires integer(n) <= m
  ensures current(trace(a0,
      for ⌊counter(0,1,n)⌋, ⌊counter(0,1,n)⌋ do a, k, i ::
        case k + 1 < m  => a[k + 1 := F(k)][k := G(k)]
        case m == k + 1 => a[k := G(k)]))[j]
    == if 0 <= j && j < integer(n) then G(j)
       else if j == integer(n) && 0 < j && j < m
       then F(j - 1) else a0[j]
```

Reasoning steps.

(1) The lemma matches with $m = SIZE$ and $F = G = (z) \mapsto z$; its side condition $n \le SIZE$ makes the guards cover all iterations.
(2) For $0 \le i < SIZE = n$, the cell is $a[i] = i$, and the assertion $i \le i$ holds.

The same lemma closes `rew`, under `forall` and `min`.

7 Results

Table 1: Proved tests out of the 125 SV-COMP tests.

	MonoCera	Cuvée	with lemmas
sum (17)	10	1	17
others (108)	32	50	56
total (125)	42	51	73

The lemmas close 22 tests that Cuvée did not prove before: all 16 open tests over sums, where MonoCera's instrumentation had its main advantage, and six of the 29 tests that only MonoCera proved. MonoCera's results are those of its paper (300 s per test); Cuvée's baseline ran with version 0.35.8 and 1 min per test, the lemmas locally with version 0.35.15 and 30 s per test, only on the tests that changed. A run of all tests with the lemmas would confirm the total.

8 Limitations

A lemma that does not match. For `rewnifrev`, the loop of `rewnif` runs downwards, with the counter $I = counter(n + i, -1, n)$ of the exit value $i = -1$. The corresponding lemma for a downward counter has the same form as the trace in the goal when printed, yet it does not match. We excluded rewriting by another rule, the unused first dependency, and the representation of the literal -1; finding the cause requires tracing the matching itself.

Specificity. Several lemmas fix the number of cases and the stride. Strides 3 to 5 with per-cell cases (`pr3–pr5`), writes by inner loops (`nr*`), recurrences with polynomial closed forms (`ifeqn*`), and nondeterministic input arrays (`standard_vararg_ground`) remain open; they would need more general lemmas, e.g. with the stride as a parameter, or reasoning beyond rewriting.